

Lab 3: Dual-Redundant CAN and Real-Time Analysis

18-449/649

Carnegie Mellon University

Release: 10/04/2026

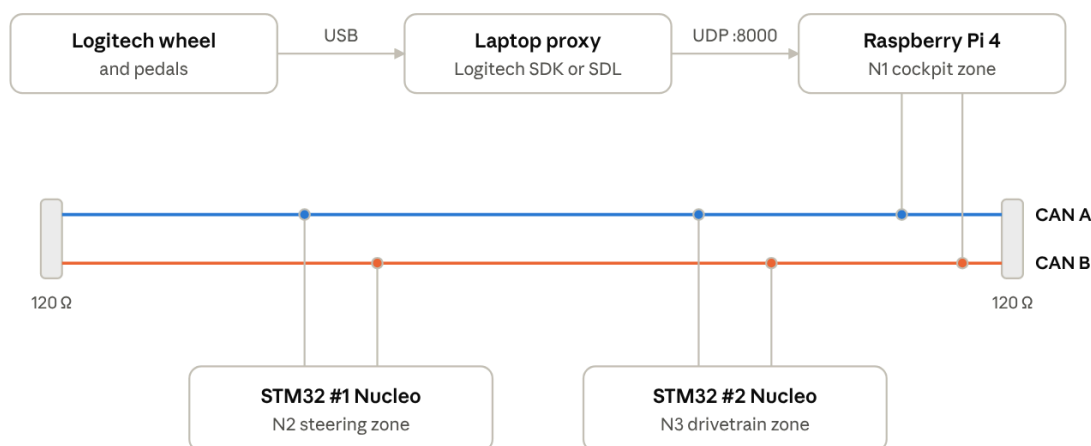
Due: 10/22/2026, 11:59pm

Version 1.0

What you are building

In this lab you split the single STM32 design from Lab 2 into the three zones from your Lab 1 design and connect them over a dual-redundant CAN bus. You will wire CAN A and CAN B between the two STM32 Nucleos and the Raspberry Pi using six XIAO CAN Bus Expansion modules, draw a task communication graph for a template set of Lab 1 requirements, prove that graph is schedulable with real-time analysis, then add the redundant bus (including the case where a message has to hop through the middle node) and show the analysis still holds or say what has to change. Finally you implement the graph in Zephyr and on the Pi and demonstrate that the system keeps running even when one set of CAN wires is removed at each node.

Three zones on two independently terminated 500 kbit/s buses



Each node reaches each bus through its own XIAO CAN module: six modules, two per node, one terminator at each end of each bus.

Figure 1: System overview

The sensors and actuators from Lab 2 are not required for this lab. The demo runs on LEDs, GPIO test points and printed values. If your Lab 2 car is still wired up you may drive it from the new zones, and the integration write-up from Lab 2 is a head start, but it is not the main focus of this lab (or graded).

What is not in this lab / deviates from the spec in your Lab 1 writeup:

- **No PCB.** The XIAO CAN modules sit on breadboards and connect to the Nucleos and the Pi with jumper wire. The per-node isolation jumpers of R4.8 are just the jumper wires on CANH / CANL - pulling one is the fault injection.

- **A template task set, not yours.** Everyone analyzes and implements the same task graph (Part 2) so that the analysis is comparable across teams. You can always go back and update your own Lab 1 design with an entry in its revision history if you come up with new ideas.
- **Force feedback is a dummy value.** The steering node transmits a force message, but its payload is a counter. However, force feedback should now work in Windows and Linux with updated drivers.

What is in the kit

Part	Why you have it
Raspberry Pi 4, supply, microSD (from Lab 2)	Node 1, the cockpit zone. Receives the UDP stream and is the CAN gateway for the wheel.
2x STM32 Nucleo-F401RE + Mini-B USB cables	Nodes 2 and 3, the steering and drivetrain zones. Each runs Zephyr and owns two CAN interfaces.
6x XIAO CAN Bus Expansion module	Two per node: one for CAN A, one for CAN B. Each is an MCP2515 controller (SPI) plus an SN65HVD230 3.3 V transceiver with a screw terminal for CANH / CANL / GND.
2x 120 Ohm resistors (or the module's P1 pad)	Bus termination, one at each end of each bus. The module has a solder pad P1 on its back that adds a 120 Ohm terminator when shorted; it ships open.
Breadboards, jumper kits, 22 AWG solid wire	One small breadboard per node for its two modules, one for the bus rails and test points. Two wire colors per bus (R4.3).
Logitech wheel, pedals, 24 V brick (from Lab 2)	Cockpit input, still through the laptop proxy.
LEDs and resistors (from Lab 2)	Four blinkers, hazard pattern, and the per-bus degraded status LEDs required by R4.7.
Small screwdrivers	The screw terminals on the modules.

The modules are 3.3 V devices with no level shifting: they connect directly to the Nucleo's 3.3 V SPI pins and to the Pi's 3.3 V GPIO header. Do not feed them 5 V. They are made to plug on top of a Seeed XIAO; here you jumper into their castellated header rows instead. Read the pinout in Part 1 before you touch them.

Part 0: Development environment

Zephyr. Zephyr ships a driver for the MCP2515 (`compatible = "microchip,mcp2515"`) that exposes it through the standard Zephyr CAN API that can be found here:

https://docs.zephyrproject.org/latest/dtoxygen/html/group_can_controller.html. You describe each module as a child of an SPI controller in a devicetree overlay for `nucleo_f401re`, with its chip-select GPIO, its INT GPIO and its crystal frequency. Two modules on one node means two children under the same SPI bus with different chip selects and different INT pins. Start from the Zephyr CAN samples (`samples/drivers/can/counter` is a good one) and the driver's devicetree binding (`dt/bindings/can/microchip,mcp2515.yaml`). You can find an interface for bindings here: <https://docs.zephyrproject.org/latest/build/dts/api/bindings/can/microchip%2Cmcp2515.html#dtbinding-microchip-mcp2515>

You can find the example here:

<https://docs.zephyrproject.org/latest/samples/drivers/can/counter/README.html#can-counter>

Appendix B has a skeleton overlay.

Raspberry Pi. Linux drives the MCP2515 through the `mcp251x` kernel driver and exposes it as a SocketCAN interface. Two lines in `/boot/firmware/config.txt` (one per module, on SPI0 CE0 and CE1), a reboot, and `ip link set can0 up type can bitrate 500000` gives you `can0` and `can1`. Install `can-utils` for `candump` and `cansend`, and `python-can` if you prefer Python for the cockpit code. Appendix B has the overlay lines.

Guiding Questions:

- What crystal is on your XIAO module? Read the marking on the metal can. The MCP2515 bit timing is derived from it, and an overlay that says 16 MHz on an 8 MHz board puts your node on the bus at half the bit rate. Both Zephyr and the Pi overlay need the real number.
- What SPI clock will you run? The MCP2515 tops out at 10 MHz. How long does one read of a received frame take over SPI at that clock, in microseconds? You will need this number in Part 3.
- The MCP2515 has one INT pin that is shared by receive, transmit-done and error interrupts. What does the Zephyr driver do in the ISR, and what does it defer to a thread? Where is that thread's priority set?

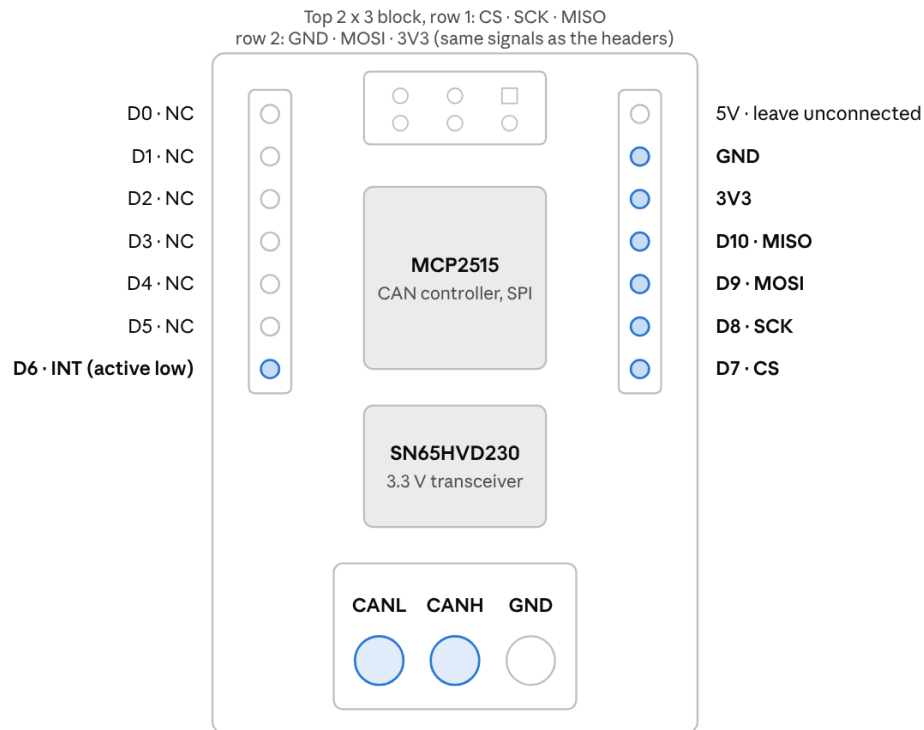
Checkpoint. Zephyr builds and flashes a CAN sample onto each Nucleo with your overlay, `can_get_state` reports the controller is active, and on the Pi `ip -details link show can0` shows state `ERROR-ACTIVE` at 500 kbit/s on both `can0` and `can1`. Do not connect any bus wire until each node comes up on its own.

Part 1: Wire up the dual CAN bus

Goal: three nodes, each with two independent CAN interfaces, on two separately terminated linear buses, and a `candump` on the Pi that sees frames from both Nucleos on both `can0` and `can1`.

The module. Each XIAO CAN module is one CAN interface: MCP2515 on SPI plus an SN65HVD230 transceiver. The pinout below is in XIAO pin names; the SPI pins are on the right header, INT is D6 on the left header, and the bus comes out of the three-position screw terminal at the bottom. Everything on the left header except D6 is unused. The six-pin block at the top is the SWD/SPI header of the XIAO footprint and also carries the SPI signals; use whichever is easier to jumper.

Six module pins carry everything: SPI on the right header, INT on the left, the bus on the terminal



Highlighted pins are the ones you wire. Pad P1 on the back adds the 120 Ω terminator when shorted; it ships open.

Figure 2: XIAO CAN module pinout, in XIAO pin names

The table maps those module pins to the Nucleo and to the Pi.

Module pin	Function	Nucleo (SPI1 on the Arduino header)	Raspberry Pi (SPI0)
3V3	Module supply	3V3	3V3 (pin 1 or 17)
GND	Ground	GND	GND
D8	SCK	D13 (PA5)	GPIO11 / SCLK (pin 23)
D9	MOSI	D11 (PA7)	GPIO10 / MOSI (pin 19)
D10	MISO	D12 (PA6)	GPIO9 / MISO (pin 21)
D7	CS	CAN A: D10 (PB6) · CAN B: D9 (PC7), or any free GPIO	CAN A: CE0 (pin 24) · CAN B: CE1 (pin 26)
D6	INT (active low)	CAN A: D2 (PA10) · CAN B: D4 (PB5), or any EXTI-capable GPIO	CAN A: GPIO25 (pin 22) · CAN B: GPIO24 (pin 18)
CANH / CANL / GND	Bus	Bus A or bus B rails on the breadboard	Bus A or bus B rails on the breadboard

The pin choices for CS and INT are suggestions; they only need to match your devicetree overlay (Nucleo) and `config.txt` (Pi). SCK, MOSI and MISO are shared between a node's two modules. CS and INT are not.

The bus. Each bus is a single linear pair: CANH and CANL run down the breadboard, every node's module taps on, and there is exactly one 120 Ohm resistor across CANH-CANL at each end. Use the module's P1 pad only on the two end nodes of a bus, or use discrete resistors on the breadboard and leave every P1 open - but not both. Measure 60 Ohm between CANH and CANL with everything powered off; 120 means one terminator, 40 means three. Use one wire color for bus A and a different one for bus B, and keep the two buses on different breadboard rails. All three nodes must share a ground: run a GND wire between the Pi, both Nucleos and the bus breadboard before you connect anything else.

Topology. The two buses are independent down to the transceiver: a node's CAN A module and CAN B module share nothing but power, ground and the SPI clock and data lines. Bringing a jumper from the bus rail to each module's screw terminal is what gives you R4.8's isolation points - pulling that jumper disconnects the node from that bus and leaves the terminator in place.

Guiding Questions:

- The SN65HVD230 has a slope-control pin (RS). How is it strapped on the module, and what does that do to your edge rates and maximum bit rate?
- Does your Nucleo's SPI1 default pin set collide with anything else you already use (the user LED on PA5 does)? Where would you move it?
- Which end of each bus gets the terminator, and what happens to the bus when you pull the jumper of an end node that is also providing termination?
- The Pi runs the MCP2515 through a kernel driver with its own interrupt thread. Who else is on SPI0, and can the two modules share the bus without one starving the other?

Checkpoint. With one node sending a frame every 100 ms on both of its modules, `candump can0` and `candump can1` on the Pi both show it. Repeat from each Nucleo. Then pull the bus A jumper of one node and confirm bus B traffic continues and bus A still works between the other two nodes. Scope CANH and CANL of each bus and show the differential idle level and a frame.

Part 2: The template task set and task communication graph

Everyone builds and analyzes the same template task set described below.. It is a subset of your Lab 1 requirements - enough to carry the brake, throttle, steering, blinker, heartbeat and error paths across CAN - with periods and deadlines fixed by the handout. If you decide to change your Lab 1 design (IDs, who is the blinker master, where the PID loop lives, etc), feel free to update your Lab 1 document and record it in a revision history at the end.

Requirements carried by this lab. R2.1 / R2.2 (throttle and brake within 2 ms, end to end), R2.3 (steering within 50 ms), R2.4 (blinker starts within 100 ms; front/rear edges within 1 ms), R2.5 (test button: local fail within 10 ms, system error within 100 ms), R4.4 (control message on every UDP update), R4.5 (heartbeat every 20 ms $\pm 10\%$, three missed on both buses = failed zone), R4.7 (hot/parallel redundancy, per-bus degraded LED).

Fixed inputs. The proxy runs at `--interval-ms 20`, so a UDP state update and therefore a CTRL message arrives every 20 ms. This is a deliberate simplification: one common 20 ms period for the control message and the heartbeats keeps the interference terms in Part 3 small enough to check by hand. A real cockpit sends at whatever rate the proxy runs, and the analysis then has to carry two periods. Both buses are 500 kbit/s, so one bit is 2 μ s and the longest standard frame (8 data bytes, worst-case stuffing, 135 bits) occupies the bus for 270 μ s.

Message catalog (template)

All frames are standard 11-bit identifiers, DLC 8, transmitted on both buses. Byte 0 of every payload is the 4-bit sequence counter (low nibble) and 4 bits of flags; the receiver uses (ID, seq) to drop the duplicate copy from the other bus.

ID	Name	From	To	Trigger / period	Deadline	Payload bytes 1-7
0x010	CTRL	N1	N2, N3	every UDP update, 20 ms	2 ms (brake, throttle path)	steer int16, throttle u8, brake u8, buttons u8, CRC-8
0x020	FAULT	any	all	on zone fail / out-of-range value; min interval 20 ms	100 ms	failing zone id, reason code
0x100	HB_N1	N1	N2, N3	20 ms	20 ms	zone state, bus A/B health bits
0x101	HB_N2	N2	N1, N3	20 ms	20 ms	zone state, bus A/B health bits
0x102	HB_N3	N3	N1, N2	20 ms	20 ms	zone state, bus A/B health bits
0x200	BLINK	N1	N2, N3	on blinker state change and every 500 ms toggle; min interval 20 ms	100 ms to start; edges of N2 and N3 within 1 ms of each other	blinker state (off / L / R / hazard), phase bit
0x300	FORCE	N2	N1	50 ms	50 ms	force int8 (dummy counter this lab)

The cockpit is the blinker master: it sends the toggle edge, and both STM32 nodes drive their blinkers on reception. The 1 ms front/rear synchronization therefore becomes a bound on the difference between the two nodes' receive response times, which is something you can compute.

Task set (template)

Priorities are yours to assign; deadline-monotonic is an obvious starting point. The WCET column holds placeholder budgets for your first pass of the analysis; replace them with values you measured on your own node with a GPIO toggle around the work (Part 3). Zephyr's MCP2515 driver handles the INT line in an ISR and does the SPI frame read in its own work thread, so the receive path on each STM32 is ISR + driver thread + your dispatch; the table folds those into CAN_RX.

Node	Task	Kind	Period / min interarrival	Deadline	WCET placeholder	Talks to
N1 (Pi)	UDP_RX	event (socket)	20 ms	-	measured	CTRL_TX, BLINK_TX
N1	CTRL_TX	event	20 ms	2 ms budget: ≤ 0.5 ms	measured	can0, can1
N1	BLINK_TX	event + 500 ms timer	20 ms	-	measured	can0, can1
N1	HB_TX	timer	20 ms	20 ms	measured	can0, can1
N1	HB_MON / FORCE_RX	socket + timer	20 ms	20 ms	measured	state, proxy UDP
N2 (STM32 front)	CAN_RX_A, CAN_RX_B	ISR + driver thread	20 ms per message, 7 msgs per bus	0.3 ms	80 μ s per frame	dispatch
N2	STEER	on CTRL	20 ms	50 ms end to end; local 1 ms	50 μ s	servo PWM
N2	BLINK_F	on BLINK	20 ms	0.5 ms from reception	20 μ s	front LEDs
N2	FORCE_TX	timer	50 ms	50 ms	120 μ s (two buses)	CAN TX A, B
N2	HB_TX	timer	20 ms	20 ms	120 μ s (two buses)	CAN TX A, B
N2	HB_MON	timer	20 ms	20 ms	30 μ s	error state, status LEDs
N2	TEST_BTN	GPIO ISR + debounce	50 ms	10 ms	20 μ s	error state, FAULT TX
N3 (STM32 rear)	CAN_RX_A, CAN_RX_B	ISR + driver thread	20 ms per message, 7 msgs per bus	0.3 ms	80 μ s per frame	dispatch
N3	DRIVE	on CTRL	20 ms	2 ms end to end; local 0.5 ms	50 μ s	motor PWM, H-bridge DIR
N3	PID	timer	10 ms	10 ms	200 μ s	motor PWM
N3	BLINK_R	on BLINK	20 ms	0.5 ms from reception	20 μ s	rear LEDs
N3	HB_TX	timer	20 ms	20 ms	120 μ s (two buses)	CAN TX A, B
N3	HB_MON	timer	20 ms	20 ms	30 μ s	error state, status LEDs

Node	Task	Kind	Period / min interarrival	Deadline	WCET placeholder	Talks to
N3	TEST_BTN	GPIO ISR + debounce	50 ms	10 ms	20 μ s	error state, FAULT TX

The Pi has no WCET column because Linux gives you no bound to put there. You will measure its UDP_RX-to-CTRL_TX latency with the test points and treat it as an input to the end-to-end analysis (Part 3).

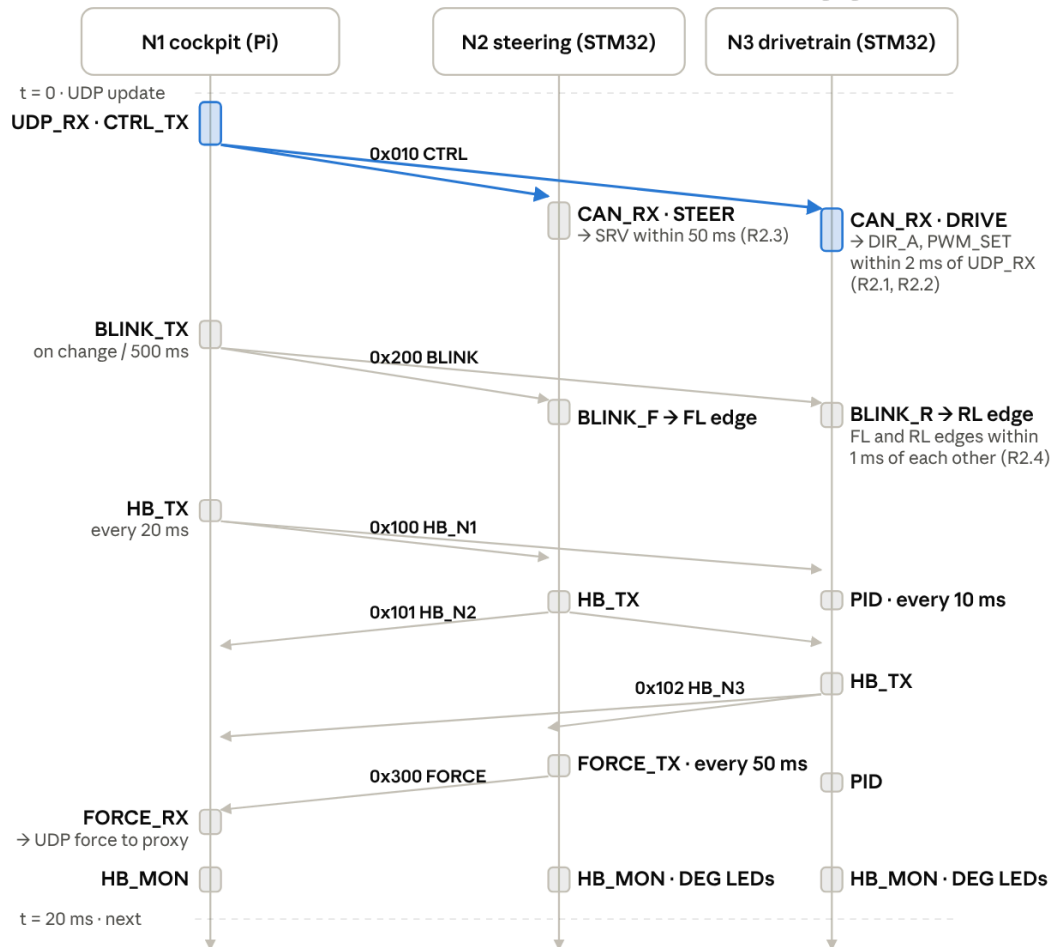
The task communication graph

A task communication graph has a vertex for every task and a directed edge for every message or shared variable between them, labeled with the period, the deadline and, for a CAN edge, the message ID.

Drawn against time it becomes a task-graph timeline like the one below: a node's work is a box on its lifeline and a message is an arrow between lifelines. The end-to-end chain for a requirement is one path through this graph, and its latency is the sum of the response times along the path.

One 20 ms cycle: every message, the processing it triggers, and its deadline

Time runs down. Boxes are task executions on a node; arrows are CAN frames. The brake chain is highlighted.



Not to scale. Every frame goes out on both buses and the receiver keeps the first copy. FAULT (0x020) is event-driven and not shown. HB_MON on each node: three heartbeats missed on both buses = failed zone; missed on one bus only = that bus degraded.

Figure 3: Example task graph, drawn as one 20 ms cycle

Figure 3 is the template task set drawn this way for one cycle: the UDP update at the top, the CTRL frame it produces, the processing that frame triggers on each STM32, the blinker and heartbeat traffic, and the force message coming back. Read down a lifeline for what a node does in order; read along an arrow for a CAN frame and who receives it. The vertical spacing is not time to scale - the point is the order and the chain of dependencies, and every requirement from Part 2 appears beside the stage that has to meet it.

Your own figure uses this form. It must show every task and message in the two tables above, with each CAN arrow labeled by ID, and the four end-to-end chains (brake/throttle, steering, blinker, error propagation) visibly traceable as paths. In Part 4 it gains the RELAY stage on N2 and the second bus hop, which in this form is simply one more box and one more arrow on the brake chain.

Guiding Questions:

- Which edges in the graph are CAN messages and which are shared memory inside one node? The two have different costs and different analysis.
- CTRL carries brake, throttle and steering with one deadline of 2 ms. Would you split it? What does a second message cost in bus time and in receive-side work?
- Where does the dedup state live, and what happens to the sequence counter when a node reboots mid-stream?
- What is the minimum interarrival of frames at one receiver? Count every message on one bus, then remember the other bus delivers a copy of each.

Checkpoint. You have a task graph figure with every task and message from the two tables above, every CAN edge labeled with its ID and period, and the four end-to-end chains (brake/throttle, steering, blinker, error propagation) highlighted.

Part 3: Real-time analysis

Goal: a written argument, with numbers, that every deadline in Part 2 is met in the worst case on a single bus. Three layers, done in order: each STM32 node on its own, each CAN bus on its own, then the end-to-end chains that cross both. Appendix A works one message and one chain all the way through.

3.1 Node schedulability (STM32, Zephyr)

Zephyr is a fixed-priority preemptive scheduler, so response-time analysis applies directly. For each task i with priority higher-than set $hp(i)$, WCET C , period T , blocking B and deadline D :

$$R_i = C_i + B_i + \sum_{j \in hp(i)} \left\lceil \frac{R_i}{T_j} \right\rceil C_j \leq D_i$$

Iterate until R stops changing. Treat ISRs as the highest-priority tasks, the MCP2515 driver's work thread as a task whose period is the shortest message period on its bus, and your own dispatch as the task it hands to. B is the longest critical section a lower-priority task can hold against you - a mutex around shared state, or the SPI bus itself, which both CAN modules and both the TX and RX paths contend for.

Guiding Questions:

- The SPI bus is a shared resource. If HB_TX is in the middle of loading a frame into module A when a CTRL frame lands on module B, how long does the DRIVE path wait? Is the SPI transaction a blocking term, interference, or both, and at what priority does the SPI driver run?
- Which Zephyr priority did you give the driver's RX thread, and is it above or below DRIVE? What goes wrong each way?
- How did you measure WCET? A GPIO toggle around the body of a thread measures execution with preemption included - how do you get the pure C_i out of it?

3.2 CAN message response times

A CAN bus is also a fixed-priority non-preemptive scheduler: arbitration picks the lowest ID, and a frame already on the wire cannot be preempted. Davis, Burns, Bril and Lukkien (2007) give the standard analysis. For message m with queuing jitter J , transmission time C , blocking B = the longest lower-priority frame that can be on the wire when m is queued (270 μ s here), and bit time T :

$$w_m = B_m + \sum_{j \in hp(m)} \left\lceil \frac{w_m + J_j + \tau_{bit}}{T_j} \right\rceil C_j$$

$$R_m = J_m + w_m + C_m \leq D_m$$

C_m for a standard frame with s data bytes, including the worst-case bit stuffing:

$$C_m = \left(47 + 8s + \left\lfloor \frac{34+8s-1}{4} \right\rfloor \right) \tau_{bit}$$

J_m is the release jitter of the sender: how late after its trigger the frame can actually be queued, which is the sender task's response time from 3.1 (and on the Pi, your measured number). Fill a table with one row per message in the catalog: J , C , B , w , R , D .

Also compute utilization per bus, $\sum C_m / T_m$ over the catalog, and check it against R4.6's 50 % ceiling.

Guiding Questions:

- The MCP2515 has three transmit buffers. If your driver queues HB, FORCE and CTRL copies at once, does the controller arbitrate by ID among its own buffers, or in buffer order? What does Zephyr's driver do, and does it change your $hp(m)$?
- Where in the end-to-end path does a frame sit after it wins arbitration and before your task sees it? The MCP2515 receive buffer, the INT line, the EXTI ISR, the SPI read. Which of these is in R_m and which is in the node analysis?
- FAULT and BLINK are sporadic. What minimum interarrival did you use, and what enforces it in code?

3.3 End-to-end chains

An end-to-end latency is the sum of the response times along a path in the task graph, plus one period of any stage that polls rather than reacts. Write each of these four chains out stage by stage and compare the total to its deadline:

Chain	Path	Deadline
Brake / throttle	UDP_RX (N1) → CTRL_TX (N1) → 0x010 on bus → CAN_RX (N3) → DRIVE (N3) → PWM / DIR change	2 ms (R2.1, R2.2)
Steering	UDP_RX (N1) → CTRL_TX (N1) → 0x010 → CAN_RX (N2) → STEER (N2) → servo PWM	50 ms (R2.3)
Blinker start	UDP_RX (N1) → BLINK_TX (N1) → 0x200 → CAN_RX (N2, N3) → BLINK_F / BLINK_R	100 ms (R2.4)
Blinker sync	0x200 arrives at N2 and N3 → BLINK_F and BLINK_R edges	difference $\leq$ 1 ms (R2.4)
Error propagation	TEST_BTN (N2) → local fail (10 ms) → HB_N2 stops → HB_MON (N1, N3) misses 3 → hazards + brake	100 ms from press (R2.5)

The Pi stage is measured, not computed: scope UDP_RX against CMD_TX for at least a minute of traffic and report the maximum and the 99th percentile, with and without a stress load on the Pi, and say which number you put in the chain and why.

Guiding Questions.

- The 2 ms brake budget has to cover a Linux socket wake-up, a SocketCAN write, bus arbitration against two other nodes' heartbeats, an EXTI interrupt, an SPI read, and a thread switch. Which of those did you not control, and what did you do about it?
- Three missed heartbeats is 60 ms. The HB_MON task on the detecting node runs every 20 ms. What is the worst-case detection latency, and does the whole error chain fit in 100 ms with your numbers?
- Blinker sync: the two STM32s receive the same frame at the same instant from the same bus. What makes their LED edges differ, and is the bound you computed symmetric?

Checkpoint. Three tables in your report - node RTA per STM32, message RTA per bus, end-to-end chains - with every row either meeting its deadline or marked as not meeting it with the reason. A missed deadline here is a result, not a failure, as long as you can say what has to change.

Part 4: Dual-redundant CAN and the relay case

Goal: extend the task graph and the analysis from Part 3 to the hot/parallel scheme of R4.7, including the case where a message cannot reach its receiver directly and has to hop through the middle node, then say whether every deadline still holds and what has to change if it does not.

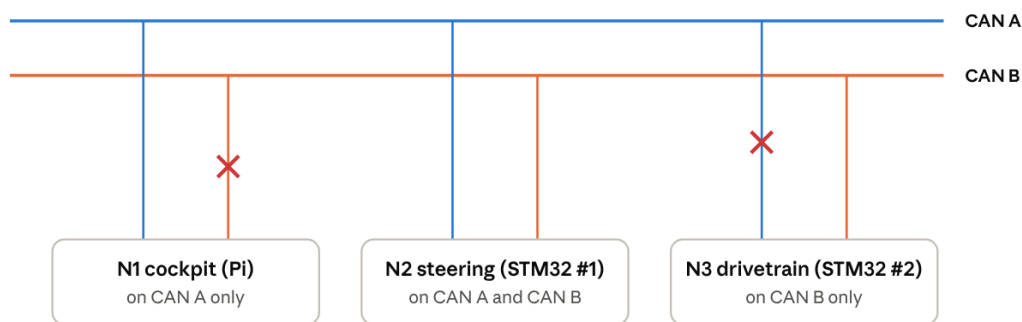
4.1 Hot/parallel transmit

Every message goes out on bus A and bus B at the same time, and every receiver takes the first valid copy and drops the second using the (ID, seq) pair. On paper the bus analysis in 3.2 does not change - each bus carries the same catalog it did before. The node analysis does: every transmitting task now does two SPI transactions instead of one, and every receiving node takes two interrupts and two SPI reads for each message, one of which is wasted. Redo 3.1 with those costs and redo the end-to-end chains. The chains should get no worse, because the receiver acts on the first copy; convince yourself of that and state the assumption it rests on.

4.2 Forwarding Messages

Now consider scenario (c) of R4.8, which is exactly the fault the topology below shows: N1 has lost CAN B and N3 has lost CAN A. N1 can only talk on A, N3 can only hear on B, and nothing N1 sends reaches N3 directly. Both buses are still alive, both nodes are only degraded (R4.7 says this is not a zone failure), and the system must stay fully operational with every R2 response time still met.

Scenario (c): N1 has lost bus B and N3 has lost bus A, so only N2 hears both



× = jumper pulled. Nothing N1 sends reaches N3 directly, so N2 must relay from A to B, and HB_N3 and FORCE the other way.

Figure 4: Dual CAN topology in R4.8 scenario (c)

The only way CTRL, BLINK and HB_N1 get from N1 to N3 is through N2, which is still on both buses. That means a new task on N2 - call it RELAY - that receives a frame on one bus and retransmits it on the other, and a new edge in the task graph: the brake chain becomes UDP_RX → CTRL_TX → 0x010 on A → CAN_RX_A (N2) → RELAY (N2) → 0x010 on B → CAN_RX_B (N3) → DRIVE. HB_N3 and FORCE have the same problem in the other direction. Extend your task graph with the RELAY task and its edges, and highlight the new brake path.

Design the relay before you analyze it. The questions you have to answer in your report:

- **Who relays, and when?** Only N2, because only N2 is on both buses, or every node that is on both buses, all the time? If every node always relays everything, what is the bus load, and how does a node tell a relayed copy from a duplicate it should drop?
- **How does a relay avoid loops?** N2 relays a frame from A to B; N3 is on B and, if it were also on A, would it relay it back? The sequence counter and a per-bus "already seen" check are the usual answer; say what yours is and what state it needs.
- **Does the relay need to know the fault exists?** A relay that always forwards costs bus bandwidth all the time but needs no detection. A relay that forwards only when it detects that a bus is degraded is cheaper but adds the detection time (R4.7: 60 ms) to the first relayed message. Which do you pick, and what does the brake chain look like during those 60 ms?
- **What priority does RELAY get on N2?** It is now on the brake path. Is it above STEER? Above the driver RX thread? What happens to N2's own deadlines?

4.3 Re-analyze

Redo all three layers with the relay in place:

1. **Node.** N2 gains RELAY; its WCET is one extra SPI transmit per relayed frame plus the dedup lookup. Its minimum interarrival is the minimum interarrival of everything N1 sends. Recompute every R on N2.
2. **Bus.** Bus B now carries the original catalog plus every relayed copy. Recompute utilization and every w_m and R_m on B; note that a relayed CTRL has the same ID as the original and therefore the same priority, but a different release jitter - it inherits R of the whole first hop.
3. **End to end.** The brake chain now has two bus traversals and one extra task response in it. Add them up. With the placeholder numbers in Part 2 it comes within a factor of two of the 2 ms budget on the STM32 side alone, before the Pi is counted. Report whether your numbers meet 2 ms, and if not, what would have to change: a shorter relay path (ISR-level forwarding?), a different priority assignment, a split of CTRL into a smaller brake-only frame, a relaxed requirement, or a decision that scenario (c) is permitted to degrade to a longer brake latency with the degradation reported. Any of these is an acceptable answer if the analysis backs it up.

Guiding Questions:

- The heartbeat rule in R4.5 says a zone is failed only when three heartbeats are missed on both buses. In scenario (c), N3 never hears HB_N1 on A and, without a relay, never hears it on B either. Does N3 declare N1 failed and enter the error state? Is that the right behavior? What does the relay change?
- The relayed HB_N1 on bus B is late by one hop. Does the 20 ms $\pm 10\%$ heartbeat tolerance survive that?
- Scenario (c) is one fault pair. Enumerate the others - which single and double jumper pulls need a relay, and which do not?

Checkpoint. Your report has the extended task graph, the three re-analysis tables for the relay case, and one paragraph stating for each of the four chains whether it still meets its deadline in scenario (c) and what you changed.

Part 5: Implement the demo

Goal: the template task set running on all three nodes, on both buses, with the relay, such that the system stays up while a TA pulls CAN jumpers. Outputs this lab are LEDs and GPIO test points: the four blinkers, a per-bus degraded LED on each node (six LEDs), and a "motor" PWM and direction pin you scope rather than connect to the H-bridge. If your Lab 2 car is still wired you may drive the real actuators, but the checkoff is scored on the test points.

Zephyr nodes. Each STM32 runs the tasks in its Part 2 table at the priorities your analysis used. The things we will look for in your code:

- Two CAN devices from devicetree, with `can_add_rx_filter` registering the catalog IDs on both and one dispatch path that dedups by (ID, seq) before any task sees a frame.
- A send helper that queues a frame on both controllers back to back and bumps the sequence counter once.
- HB_MON tracking the last-seen time of every other zone per bus, driving the degraded LEDs per R4.7 and the error state per R4.5, with the thresholds in one place.
- RELAY on N2 (and, if you chose that design, N3) implemented as its own thread with a priority you can point at.
- The error state machine from Lab 1: power-up in error, out when heartbeats from both other zones are seen on at least one bus and a valid CTRL has arrived; in on test-button, out-of-range CTRL, or three missed heartbeats on both buses.
- The independent watchdog enabled on each STM32 (R4.5).
- No SPI access, logging or allocation inside an ISR, and the shared state between the dispatch path and the actuator tasks protected by something that is not `volatile`.

Raspberry Pi. The cockpit code decodes the UDP stream (Lab 2 Appendix A), toggles `UDP_RX`, builds CTRL and BLINK, writes each to `can0` and `can1`, toggles `CMD_TX`, sends `HB_N1` every 20 ms, and reads heartbeats and FORCE from both interfaces. C with `SocketCAN` or Python with `python-can` are both fine. Pin the process to a core and give it `SCHED_FIFO` if your measurements in Part 3 need it, and say so in the report.

Bad input. CTRL values outside the ranges in your team-defined values table must put the receiving zone into the error state and send `FAULT`. Build a way to inject one - a flag on the Pi that corrupts the next frame, or `cansend` from a laptop on the bus.

Guiding Questions:

- Where does the sequence counter live on the Pi, and is it the same counter for CTRL, BLINK and `HB_N1` or one per ID? What does the receiver assume?
- What does your node do when a frame arrives on bus A with a sequence number it has already seen from bus A? That is not a duplicate.
- A TA pulls N2 off bus A while N2 is mid-transmission. The MCP2515 will go error-passive and may go bus-off. What does the Zephyr driver report, and how does your node recover within 100 ms of the jumper going back in (R4.8)?

Sanity check. With everything running, `candump -c any` on the Pi shows every ID in the catalog on both interfaces at its catalog rate. Pull one jumper: the matching degraded LED comes on within 60 ms, every blinker and the servo output keep following the wheel. Put it back: the LED goes off within 100 ms.

Test points and fault injection

As in Lab 2, bring every test point out to one small breadboard per node, one signal per row, in the order below, labeled on tape beside the board. We clip onto this board and nothing else.

Row	Node 1 (Pi)	Node 2 (STM32 front)	Node 3 (STM32 rear)
1	GND	GND	GND
2	GND	GND	GND
3	UDP_RX	CAN_RX_A	CAN_RX_A
4	CMD_TX	CAN_RX_B	CAN_RX_B
5	CAN_TX_A	CAN_TX_A	CAN_TX_A
6	CAN_TX_B	CAN_TX_B	CAN_TX_B
7	DEG_A (LED)	DEG_A (LED)	DEG_A (LED)
8	DEG_B (LED)	DEG_B (LED)	DEG_B (LED)
9	-	SRV	PWM_SET
10	-	FL	PWM_OUT
11	-	FR	DIR_A
12	-	RELAY_TX	RL
13	-	ERR	RR

Point	Toggles on
UDP_RX, CMD_TX	As in Lab 2: reception of a UDP update, transmission of the command (now the CAN write).
CAN_TX_A / CAN_TX_B	The call that queues a frame on that controller.
CAN_RX_A / CAN_RX_B	Your dispatch code seeing a frame from that controller, after the SPI read.
RELAY_TX	The relay queuing a forwarded frame on the other bus.
DEG_A / DEG_B	The per-bus degraded status LED (R4.7); doubles as a test point.
ERR	High while the zone is in the error state.
SRV, PWM_SET, PWM_OUT, DIR_A, FL, FR, RL, RR	As in Lab 2.

Bus probe points. Each bus has a row for CANH and a row for CANL on the bus breadboard, labeled A_H, A_L, B_H, B_L, so we can scope the differential signal and so each node's jumper from the bus row to its module's screw terminal is reachable. Those jumpers are the fault injection: the TA pulls the A or B jumper of any node, in any combination, with the system running.

Measuring the chains. Trigger on UDP_RX, probe CAN_TX_A on N1 and CAN_RX_A on N3 and DIR_A on N3, and you have the Pi stage, the bus stage and the node stage of the brake chain on one capture. For the relay case, pull N1's B jumper and N3's A jumper and move the second probe to RELAY_TX on N2. For blinker sync, probe FL on N2 and RL on N3 together and measure the edge offset over at least ten edges. Report the maximum, not the typical, for everything.

Which Lab 1 requirements apply

Lab 1 requirement	This lab
R1 - three zones, preemptive RTOS, independent clocks	Yes. Two STM32 zones on Zephyr, Pi as cockpit.
R2.1 / R2.2 - throttle and brake within 2 ms	Yes, end to end from UDP_RX on the Pi to DIR_A / PWM_SET on N3, over CAN. Analyzed in Part 3 and measured at checkoff. If your analysis says it cannot be met in the relay case, the report says so and why.
R2.3 - steering within 50 ms	Yes, to SRV on N2. Force feedback payload is a dummy.
R2.4 - blinkers within 100 ms, 1 Hz $\pm 10\%$, front/rear within 1 ms	Yes. This is the first lab where the two halves are on different MCUs with different clocks; the BLINK message is the sync strategy.
R2.5 - test button, local fail within 10 ms, system error within 100 ms	Yes, a hardware button on each STM32 and a wheel button for the Pi.
R3 - error state	Yes, including power-up in error until heartbeats and a valid CTRL arrive.
R4.1 - R4.3 - dual CAN, 500 kbit/s, termination, common ground	Yes, on breadboards instead of a PCB.
R4.4 - message catalog with sequence counter	Yes, the template catalog in Part 2.
R4.5 - heartbeat and fault detection	Yes.
R4.6 - utilization $\leq 50\%$, CAN test headers, TX / RX test points	Yes.
R4.7 - hot/parallel redundancy, degraded LEDs	Yes.
R4.8 - fault-injection jumpers, scenarios (a)-(f)	Yes, with breadboard jumpers in place of header jumpers. All six scenarios are demonstrated.
S3 - force feedback	No. Dummy payload only.
S6 - CAN fault tolerance	Yes, this is the lab.
Lab 2 sensors and actuators	Optional. Test points stand in for the H-bridge and servo.

Checkoff

Run the following sequence for a TA with your system in front of them.

1. **Cold start.** Power up with the proxy stopped: all three nodes in the error state, hazards flashing, ERR high, no degraded LEDs. Start the proxy; the system leaves the error state once heartbeats and a valid CTRL arrive.
2. **Bus traffic.** `candump -c any` on the Pi shows every catalog ID on both interfaces at its rate. We will read the sequence counters.
3. **Steering and blinkers.** Wheel full left, center, full right tracks on SRV. Left, right, cancel on the blinkers; FL and RL edges on the scope within 1 ms over ten edges.
4. **Brake timing.** Scope capture of the brake chain: UDP_RX to DIR_A, with the Pi, bus and node stages identifiable.
5. **Single-bus faults (R4.8 a, b).** Pull any one node's A jumper: its DEG_A comes on within 60 ms, nothing else changes. Restore it: DEG_A off within 100 ms. Same for B. We pick the node.
6. **Whole-bus loss (R4.8 e).** Pull every node off bus A. All DEG_A on, system fully operational on B. Restore.
7. **The hop (R4.8 c).** Pull N1 off B and N3 off A. DEG_B on N1, DEG_A on N3, RELAY_TX toggling on N2, and the wheel still steers, blinks and brakes. Scope capture of the relayed brake chain: UDP_RX to RELAY_TX to DIR_A. We will compare it to the number in your report.
8. **Isolation (R4.8 d).** Pull both jumpers of one STM32. Within 100 ms: that node in its fail-safe state, the other two in the system error state. Restore both: normal operation within 500 ms, no reset.
9. **Test button.** Single press on N2: hazards and braked motor on N3 within 100 ms, ERR high on all three. Double press: back to normal. Then the wheel's test button for the cockpit zone.
10. **Bad input.** Inject an out-of-range CTRL; the receiving zone enters the error state and sends FAULT.
11. **Analysis walk-through.** One node RTA row, one message RTA row and the relayed brake chain, explained by a team member who did not write that part of the report.

Expect verbal questions throughout, directed at everyone on the team. They will be about the design decisions in Parts 2 through 5 and about the numbers in your analysis - the guiding questions are a fair preview.

Deliverables

The grade is split between the system working in front of us and the written analysis that predicted it would.

Demo	Points
Live demo - the checkoff sequence above	40
Checkoff questions, answered by the team	20
Report (to be submitted to Gradescope)	
Wiring schematic of the dual bus: both buses, termination, every module's SPI / INT / CS pins, test-point boards	6
Task communication graph figure: every task and message from Part 2, CAN edges labeled with ID and period, the four chains highlighted	8
Single-bus analysis: node RTA tables for N2 and N3, message RTA table per bus with utilization, end-to-end chain table, measured Pi latency	10
Dual-bus and relay analysis: extended graph, relay design decisions, re-analysis tables for scenario (c), verdict per chain and what changed	10
Fault-injection matrix (R4.8 a-f): expected state of each zone and LED, measured recovery time	4
Updated team-defined values table and Lab 1 revision-history entry	2

The report should be no more than six pages excluding the schematic and scope captures. Tables of numbers are expected; prose should be the argument that connects them. Every WCET and latency you use must say whether it was measured or assumed, and a measured number must say how.

Update your Lab 1 design document to match the template catalog and task set, with an entry in its revision history. We may ask for it at checkoff.

Appendix A: Worked example with the placeholder numbers

These use the WCET placeholders from Part 2 and an assumed Pi latency. They show the shape of the work; your report uses your measured numbers.

Frame time. Every catalog frame is DLC 8: $C_m = (47 + 64 + \text{floor}(97/4)) \times 2 \mu\text{s} = (111 + 24) \times 2 \mu\text{s} = 270 \mu\text{s}$. Bus utilization on one bus, with FAULT and BLINK at their 20 ms minimum interarrival: $(\text{CTRL} + \text{FAULT} + 3 \times \text{HB} + \text{BLINK}) / 20 \text{ ms} + \text{FORCE} / 50 \text{ ms} = 6 \times 270 / 20000 + 270 / 50000 = 8.1 \% + 0.5 \% = 8.6 \%$, well under 50 %.

Message 0x102 HB_N3. Four higher-priority IDs (0x010, 0x020, 0x100, 0x101), one lower one on the wire at worst, so $B = 270 \mu\text{s}$. First pass $w = 270 + 4 \times 270 = 1350 \mu\text{s}$; second pass every $\text{ceil}((1350 + J + 2) / 20000)$ is still 1, so w stays $1350 \mu\text{s}$. $R = J + 1350 + 270 = J + 1620 \mu\text{s}$, against a 20 ms deadline. With $J = 150 \mu\text{s}$ (the HB_TX response time on N3) that is 1.77 ms.

Message 0x010 CTRL. Highest ID, so no interference, only blocking: $R = J + 270 + 270 = J + 540 \mu\text{s}$, where J is the Pi's UDP_RX-to-CTRL_TX latency.

Brake chain, single bus. Pi stage (measured, say $300 \mu\text{s}$ typical, 1.5 ms maximum under load) + bus $540 \mu\text{s}$ + CAN_RX on N3 (ISR, driver thread, SPI read, dispatch: about $150 \mu\text{s}$ response) + DRIVE (about $250 \mu\text{s}$ response with blocking) $\approx \text{Pi} + 0.94 \text{ ms}$. With the typical Pi number that is 1.24 ms, inside 2 ms; with the maximum it is 2.44 ms, outside. The analysis therefore says: the STM32 side fits, and the requirement is met only if the Pi stage is bounded below about 1 ms. That sentence, with your numbers, is what we want in the report.

Brake chain, relay case. Add CAN_RX on N2 ($150 \mu\text{s}$) + RELAY on N2 (about $200 \mu\text{s}$ response) + a second bus traversal on B ($540 \mu\text{s}$): $\text{Pi} + 1.83 \text{ ms}$. The STM32 and bus side alone is now within 10 % of the 2 ms budget and the Pi has no room. Something changes; Part 4.3 lists the options.

Appendix B: Configuration starting points

Zephyr overlay for nucleo_f401re (boards/nucleo_f401re.overlay in your app). Check every index against dts/bindings/can/microchip,mcp2515.yaml and the board's arduino_header gpio-map before you trust it; the osc-freq must match the crystal on your module.

```
/ {
aliases {
can-a = &can0;
};
};

&spi1 {
status = "okay";
cs-gpios = <&gpio0 6 (GPIO_ACTIVE_LOW | GPIO_PULL_UP)>;

can0: mcp2515@0 {
compatible = "microchip,mcp2515";
// What should reg be?
reg = <>;
// What should the maximum frequency be?
spi-max-frequency = <>;
```

```
osc-freq = <16000000>;
status = "okay";
label = "can0";
};

// other can chip...
};
```

Raspberry Pi (/boot/firmware/config.txt, then reboot):

```
dtparam=spi=on
dtoverlay=mcp2515-can0,oscillator=16000000,interrupt=25
dtoverlay=mcp2515-can1,oscillator=16000000,interrupt=24
```

mcp2515-can0 is on SPI0 CEO and mcp2515-can1 on CE1. Bring them up and watch them:

```
sudo apt install can-utils
sudo ip link set can0 up type can bitrate 500000
sudo ip link set can1 up type can bitrate 500000
ip -details link show can0
candump -c -t d any
cansend can0 010#0F00FF00000000A5
```

In Python, pip install python-can and can.Bus(channel="can0", interface="socketcan"). The Zephyr documentation for the CAN API and the MCP2515 binding, plus the Lab 2 Appendix B material, cover the rest.

Appendix C: Things that often go wrong

Symptom	Usually means
A node transmits but nothing else sees it; its TX LED stays solid; controller goes error-passive	No other node acknowledging. Bit rate mismatch (wrong <code>osc-freq</code> or <code>oscillator</code> for the crystal), or the node is the only one on that bus.
Frames received at exactly half or double the expected rate, garbage payloads	Oscillator setting is 8 vs 16 MHz wrong on one node.
Everything works with two nodes, fails when the third is added	Termination. Measure CANH-CANL resistance powered off: 60 Ohm is right.
Random error frames, works at 125 kbit/s but not 500	Long unshielded jumpers, missing common ground, or RS pin slope control. Shorten the bus, twist the pair.
Pi sees <code>can0</code> but <code>can1</code> never comes up	The second overlay's interrupt GPIO is wrong or shared, or CE1 is not wired. <code>`dmesg</code>
Zephyr <code>can_send</code> returns <code>-EAGAIN</code> or hangs	All three MCP2515 TX buffers full because the bus is not acknowledging; see the first row.
Node stays in error state after power-up forever	Waiting for heartbeats from both other zones on at least one bus, plus a valid CTRL. Check which one is missing with <code>candump</code> .
Degraded LED flickers on a healthy bus	Threshold too close to the heartbeat period, or <code>HB_MON</code> jitter. Three periods (60 ms) with margin.
Pulling a jumper takes a node to bus-off and it never recovers	Automatic bus-off recovery is off, or your code never calls <code>can_recover</code> . R4.8 wants recovery within 100 ms of restoration.
Relayed brake chain much slower than computed	RELAY below the driver RX thread or below STEER in priority, or the relay waiting on a mutex held by <code>HB_TX</code> across both SPI transactions.
Blinker edges on N2 and N3 drift apart over a minute	One node is toggling on its own timer instead of on the BLINK message.
STM32 stops enumerating after wiring	See "How to destroy an STM32" in Lab 2.

Revision History

10-02-26 Version 1.0: Initial release to the class